

Lab 2: Verification using Vehicle

AI Security

Due: Monday 28 September, 23:59

Overview

The purpose of this assignment is to work through the Vehicle tutorial and apply its methods to a model and dataset of your own. The tutorial is available at <https://vehicle-lang.github.io/tutorial/>, and the accompanying reference code is at <https://github.com/vehicle-lang/tutorial>. You are expected to follow the tutorial first and then adapt its examples, rather than to write everything from scratch. Kaggle (<https://www.kaggle.com/datasets>) is a common website to find datasets and to find a dataset that interests you. I would choose a dataset with a modest number of input features, since verification time grows quickly.

1 Specification and Robustness Testing

Select a dataset that we haven't used in class and that is not used in the Vehicle tutorial and train a simple classifier on it. We recommend that you choose a simple classifier similar to the ones used in the examples. Export the trained model to ONNX.

1. Train your model and export it to a `.onnx` file. ✓
2. Write a Vehicle specification containing two properties:
 - (a) an ε -ball classification robustness property; ✓
 - (b) one property that is not a robustness property. The property should be motivated by the dataset you have chosen. ✓ *I chose to define a property that is whether high alcohol content wine is classified as good. I am unsure if this is what was intended?*
3. Verify both properties using Marabou. Repeat the verification for at least three values of ε . ✓
4. Train your model for several different numbers of epochs and repeat the verification for each. Report accuracy, loss, the values of ε used, and the number of instances for which the property was verified. ✓

Before commencing my answers, i just want to note that the latter parts on my work on this lab (including writing these answers) were done in short time as i had already spent around 10 hours getting stuff to run, then having to redesign my ML model only to see no improvement and trying to do various maneuvers to create a decent classifier with no success. This hand-in reflects what I have been able to do in 10-12 hours of work on the lab.

Model	Epochs	Accuracy	Loss	ε	Correctly Classified	Verified	Falsified	Alcohol Property Verified	Alcohol Property Falsified
Wine MLP	5	40.4%	1.756	0.005	12/50	12/50	38/50		
	5	40.4%	1.756	0.01	12/50	12/50	38/50		
	5	40.4%	1.756	0.05	12/50	12/50	38/50		
Wine MLP	25	56.2%	1.102	0.005	32/50	30/50	20/50		
	25	56.2%	1.102	0.01	32/50	29/50	21/50		
	25	56.2%	1.102	0.05	32/50	18/50	32/50		
Wine MLP	50	54.2%	1.073	0.005	26/50	26/50	24/50		
	50	54.2%	1.073	0.01	26/50	26/50	24/50		
	50	54.2%	1.073	0.05	26/50	16/50	34/50		
Wine MLP	100	60.4%	0.948	0.005	31/50	29/50	21/50		
	100	60.4%	0.948	0.01	31/50	26/50	24/50		
	100	60.4%	0.948	0.05	31/50	15/50	35/50		
Wine MLP	150	62.0%	0.908	0.005	31/50	30/50	20/50		
	150	62.0%	0.908	0.01	31/50	28/50	22/50		
	150	62.0%	0.908	0.05	31/50	7/50	43/50		
Wine MLP 11-32-16-8	50	61.2%	0.940	0.005	30/50	30/50	20/50		
	50	61.2%	0.940	0.01	30/50	26/50	24/50		
	50	61.2%	0.940	0.05	30/50	14/50	36/50		
Wine MLP 11-32-16-8	100	63.0%	0.911	0.005	32/50	31/50	19/50		
	100	63.0%	0.911	0.01	32/50	28/50	22/50		
	100	63.0%	0.911	0.05	32/50	5/50	45/50		
Wine MLP 11-32-16-8	150	63.6%	0.886	0.005	32/50	31/50	19/50	5/9	4/9
	150	63.6%	0.886	0.01	32/50	30/50	20/50	5/9	4/9
	150	63.6%	0.886	0.05	32/50	7/50	43/50	5/9	4/9
Wine MLP robust	25	56.8%	0.516	0.005	30/50	28/50	22/50	6/9	3/9
	25	56.8%	0.516	0.01	30/50	28/50	22/50	6/9	3/9
	25	56.8%	0.516	0.05	30/50	16/50	34/50	6/9	3/9

Discussion

Using your results, address the following.

1. How does the largest verifiable ε change as you train for longer? *The largest "verifiable" ε becomes smaller as the number of epochs increases. (even though in our experiment, we perform pretty poor in general) We particularly see that the difference between the # of correctly classified examples and # of verifiable examples grows, as the number of epochs increases. Additionally, at higher epochs, increasing ε gives a sharper decrease in the # of verifiable examples.*
2. Describe why verification outcomes degrade as ε grows. Report how verification time varied. *Verification outcomes degrade as ε grows because the ε -ball around the input becomes larger. This means that the neighborhood of the perturbations becomes larger and may overlap with another class. In higher epochs, the boundaries of the classes generally tend to fit closer to the example data, making perturbations in larger neighborhoods more likely to be classified incorrectly. Verification time varies because the number of examples to check grows as ε increases.*
3. Your robustness property is tested on specific instances from your dataset. What does verifying those instances tell you about inputs your model will encounter in deployment? *Verifying those instances tells us that the model is not robust to perturbations in the neighborhood of the input. This means that the model may be vulnerable to adversarial attacks in the neighborhood of the up-until-now observed inputs. We do not have any test data that is not used to fit the model, so we cannot know if the model is robust to perturbations in the neighborhood of unseen inputs that may encounter when deployed.*
4. Determine the largest ε that would verify all examples and the smallest ε at which you could find a counterexample. Comment on the gap between the two. *The largest ε that would verify all examples is 0.05 at 5 epochs, but this also constitutes a really bad classifier. The smallest ε at which we could find a counterexample is 0.005 at 25 epochs. I am unsure exactly how to answer this question in the context of my experiment as the classifier is performing rather poorly, and i therefore do not think i will reach the desired reflections.*
5. For your non-robustness property: describe a failure or attack that violating it would enable. *The non-robustness property is a little bit gimmicky, and describes a commonly believed notion among young people that high alcohol content wine is good. (no comment on low alcohol being bad) If a network gets this verified, it will generally predict high alcohol content wine as good, thereby confirming young people in the skewed belief systems.*

Another property i considered which is less gimmicky and more mathematical, would be to check that the model is not too volatile in a positive direction on any single feature. I did not get around to defining this property.

2 Robustness Variants and Counterexample Validation

1. Specify and verify one further robustness variant, for example standard robustness, strong classification robustness, or Lipschitz robustness. State the definition you are using and explain how it differs from the property in Section 1. *I specified and verified standard robustness. Standard robustness is different from classification robustness (i.e. ε -ball classification robustness) in that it allows for the output scores to change by up to δ , rather than just the label being the same for two inputs that are within ε of each other. It is much stricter, as we require full output vectors to be within δ of each other, rather than just the labels (i.e. the argmax of the vector). In this case, when we increase ε , the number of verifiable examples*

increases, as the ε -ball around the input becomes larger, and opens the door for the output space to grow as well.

Note: Outputs here were extracted with short time to go, they are LLM extracted and may not be completely correct.

2. Take one counterexample returned by Marabou and evaluate it on your network directly. Include the input and the resulting output scores in your report.

Submission

Submit the following via learnIT by the deadline given above:

- your code, including all Vehicle specification files;
- the trained model in ONNX format;
- dataset and .py files you used to train and evaluate your model;
- a report in PDF format addressing the tasks in Sections 1 and 2.

An oral assessment will be held during the exercise session on 29 September. If you are unable to attend that session, contact the TA at samgha@itu.dk in advance and we'll figure out another time.